

12주차

웹과 DB 연동 1 (조회)

Node.js·Express·mysql2로 상품 목록 띄우기

이번 주를 마치면

목표 ①	브라우저-웹 서버-DBMS 3계층 구조와 각 계층의 역할 설명
목표 ②	Node.js 프로젝트에 Express·mysql2를 설치해 daon DB에 연결
목표 ③	SELECT 결과를 HTML 표로 출력하는 조회 페이지 만들기
목표 ④	쿼리스트링으로 검색 조건을 받아 WHERE 절에 반영
목표 ⑤	SQL 인젝션의 원리와 파라미터 바인딩(?)의 필요성 설명

다운유통 상품 목록을 웹 페이지에 띄웁니다

오늘 다루는 다섯 가지

1 웹과 데이터베이스가 만나는 구조

2 실습 환경 준비 — Node.js · Express · mysql2

3 첫 조회 페이지 — 상품 목록을 웹 표로

4 검색 조건 추가 — 쿼리스트링과 WHERE

5 파라미터 바인딩 — ? 하나가 회사를 지킨다

3계층(3-Tier) 아키텍처

계층	담당	하는 일
① 프레젠테이션 계층	브라우저 (크롬 등)	화면 표시, 사용자 입력을 웹 서버로 전송
② 애플리케이션 계층	웹 서버 (Node.js + Express)	요청을 받아 SQL 실행, 결과를 HTML로 조립해 응답
③ 데이터 계층	DBMS (MySQL)	데이터 저장·조회, 무결성·권한 관리

사내 재고 조회·쇼핑몰 관리자·회원 관리 — 거의 모든 업무 화면이 이 구조 위에서 돌아간다

고객은 SQL을 모른다 — 웹 서버가 고객 대신 SELECT를 실행하고 결과를 화면으로 바꿔 보여 준다

1. 웹과 데이터베이스가 만나는 구조

주소 하나를 열면 일어나는 다섯 단계

예: 브라우저에서 `http://localhost:3000/products` 를 연다



이 수업의 웹 서버 = 자바스크립트 기반 Node.js + Express (Java Spring·Python Django 등도 같은 구조)

브라우저가 DB에 직접 접속하지 않는 이유

접속 정보 노출

01

DB 비밀번호가
방문자에게 그대로 드러난다

권한 통제 불가

02

누가 어떤 SQL을 실행할지
막을 방법이 없다

연결 폭증

03

이용자 수만큼
DB 연결이 늘어난다

10주차 GRANT가 여기서 빛을 발한다 — 웹 서버용 DB 계정에 필요한 권한만 주면 피해 범위가 제한된다

2. 실습 환경 준비

설치 확인과 프로젝트 만들기

하려는 일	터미널 명령	참고
Node.js 설치 확인	node -v	버전 번호가 나오면 설치됨
npm 설치 확인	npm -v	Node.js와 함께 설치된다
프로젝트 폴더 생성	mkdir daon-web	
폴더로 이동	cd daon-web	
프로젝트 시작	npm init -y	package.json 생성
패키지 설치	npm install express mysql2	node_modules 생성

버전이 안 나오면 nodejs.org에서 LTS 설치 후 터미널을 새로 연다 — 윈도우는 명령 프롬프트 또는 PowerShell

설치한 두 패키지의 역할

01

express

웹 서버 프레임워크
주소(라우트)별로 실행할 코드를 연결

02

mysql2

Node.js에서 MySQL에 접속해
SQL을 실행하는 드라이버

설치 후 생기는 `node_modules(설치된 패키지)·package.json(프로젝트 설정)`은 지우거나 고치지 않고 그대로 둔다

2. 실습 환경 준비

db-test.js — daon 데이터베이스 연결 확인

```
1  const mysql = require('mysql2/promise');
2
3  async function main() {
4    const conn = await mysql.createConnection({
5      host: 'localhost',          // MySQL 서버 위치 - 내 컴퓨터
6      user: 'root',              // 접속 계정
7      password: '본인 비밀번호', // ← 이 줄만 본인 환경에 맞게 수정
8      database: 'daon',          // 사용할 데이터베이스 이름 (USE daon; 과 같은 역할)
9    });
10
11   const [rows] = await conn.query('SELECT COUNT(*) AS cnt FROM products');
12   console.log('상품 수:', rows[0].cnt); // 상품 수: 10
13
14   await conn.end();
15 }
16
17 main();
```

1

1행

- 1** **mysql2 불러오기**
/promise = async/await 문법으로 쓴다

2

4-9행

- 2** **DB에 연결**
접속 정보는 Workbench와 같다 — 비밀번호 줄만 수정

3

11-12행

- 3** **SQL 실행과 결과**
[rows] = 결과의 첫 칸(행들) · rows[0].cnt = AS 별칭

4

14행

- 4** **연결 닫기**
닫지 않으면 프로그램이 끝나지 않는다

daon-web 폴더에 db-test.js로 저장 → 터미널에서 node db-test.js → 「상품 수: 10」이면 성공

3. 첫 조회 페이지

app.js ① 패키지와 커넥션 풀

```
1 const express = require('express'); // 웹 서버 프레임워크
2 const mysql = require('mysql2/promise'); // MySQL 드라이버 (async/await 버전)
3
4 const app = express();
5
6 const pool = mysql.createPool({
7   host: 'localhost',
8   user: 'root',
9   password: '본인 비밀번호', // ← 이 줄만 본인 환경에 맞게 수정 (실무는 환경 변수로)
10  database: 'daon',
11  connectionLimit: 10, // 동시에 유지할 연결 수 최대 10개
12  charset: 'utf8mb4', // 한글·이모지가 깨지지 않도록 문자 집합 지정
13 });
```

1

1-2행

1

패키지 두 개

웹 서버 프레임워크와 MySQL 드라이버

2

4행

2

Express 앱

웹 서버 본체 — 여기에 라우트를 붙인다

3

6-13행

3

커넥션 풀

연결을 미리 만들어 두고 돌려쓴다

요청마다 새로 접속·해제하면 느리고, 접속자가 많으면 DB가 버티지 못한다

3. 첫 조회 페이지

app.js ② /products 라우트 — SQL 결과를 표로

```
15 app.get('/products', async (req, res) => {
16   try {
17     const [rows] = await pool.query(
18       'SELECT prod_id, name, category, price, stock FROM products ORDER BY prod_id'
19     );
20
21     let html = '<meta charset="utf-8"><h1>다운유통 상품 목록</h1>';
22     html += '<table border="1" cellpadding="6">'; // 표 시작
23     html += '<tr><th>상품번호</th><th>상품명</th><th>카테고리</th><th>판매가</th><th>재고</th></tr>';
24     for (const p of rows) {
25       html += '<tr><td>' + p.prod_id + '</td><td>' + p.name + '</td><td>'
26         + p.category + '</td><td>' + p.price + '</td><td>' + p.stock + '</td></tr>';
27     }
28     html += '</table>'; // 표 끝
```

1

15행

1 라우트

이 주소로 GET 요청이 올 때만 실행

2

17-19행

2 pool.query

결과 격자의 한 줄 = rows 배열의 원소 하나

3

21-23행

3 HTML 조립

meta charset은 한글 깨짐 방지

4

24-27행

4 행마다 <tr>

p.name처럼 SELECT 컬럼명 그대로 꺼낸다

await를 빠뜨리면 rows에 결과가 아니라 「약속(Promise)」 객체가 들어가 화면이 깨진다

3. 첫 조회 페이지

app.js ③ 응답과 서버 시작

```
29     res.send(html);
30   } catch (err) {
31     res.status(500).send('조회 실패: ' + err.message);
32   }
33 });
34
35 app.listen(3000, () => {
36   console.log('서버 실행 중: http://localhost:3000/products');
37 });
```

res.send로 HTML 응답 · 실패하면 500(서버 쪽 오류) · 3000번 포트에서 대기 — 실행: node app.js

실행과 접속

하려는 일	명령 · 주소	참고
서버 켜기	node app.js	「서버 실행 중」 메시지 확인
브라우저로 접속	http://localhost:3000/products	상품 10종이 표로 나온다
서버 끄기	Ctrl+C	터미널에서 누른다
고친 코드 반영	node app.js	끄고 다시 켜야 반영된다

화면이 안 바뀌면 서버 재시작부터 확인한다

막혔을 때 먼저 보는 곳

■ 증상

! **node -v에 버전이 안 나온다**

! **db-test.js 실행 시 에러**

! **화면이 깨지거나 표가 안 나온다**

! **코드를 고쳤는데 화면이 그대로**

■ 원인

Node.js가 설치되지 않았다

비밀번호·DB 이름 오타, MySQL 서버 미실행

pool.query 앞의 await를 빠뜨렸다

서버가 고치기 전 코드로 실행 중

■ 해결

✓ **nodejs.org에서 LTS 설치 후 터미널을 새로 연다**

✓ **에러 메시지 전체를 생성형 AI에 붙여 원인을 물어본다**

✓ **await pool.query(...)로 결과를 기다린다**

✓ **Ctrl+C로 끄고 node app.js로 다시 켜다**

실무에서는 err.message를 화면에 그대로 노출하지 않는다 — SQL 구조가 드러날 수 있다

4. 검색 조건 추가

쿼리스트링 — 주소 뒤 ?이름=값

/search?category=식품 으로 접속하면 Express가 그 값을 req.query.category로 꺼내 준다



새 라우트는 app.js의 app.listen(...) 윗줄에 이어 붙인다

4. 검색 조건 추가

카테고리 검색 라우트 (app.js에 추가)

```
1 app.get('/search', async (req, res) => {
2   try {
3     const category = req.query.category; // 쿼리스트링에서 값 읽기 (예: '식품')
4
5     const [rows] = await pool.query(
6       'SELECT prod_id, name, price, stock FROM products WHERE category = ? ORDER BY prod_id',
7       [category] // ? 자리에 값이 안전하게 들어간다 (파라미터 바인딩)
8     );
9
10    let html = '<meta charset="utf-8"><h1>' + category + ' 상품</h1><ul>';
11    for (const p of rows) {
12      html += '<li>' + p.name + ' - ' + p.price + '원 (재고 ' + p.stock + ')</li>';
13    }
14    html += '</ul>';
15    res.send(html); // 브라우저로 응답
16  } catch (err) {
17    res.status(500).send('조회 실패: ' + err.message); // 실패 시 500 오류로 응답
18  }
19 });
```

3행

- 1 값 읽기
req.query.category

5-8행

- 2 ? 플레이스홀더
값은 두 번째 인자 배열로 따로

10-14행

- 3 목록
행 하나 = 한 줄

SQL 안의 ?는 '값이 들어올 자리'만 표시한다 — 문자열 이어 붙이기 금지

4. 검색 조건 추가

주소만 바꾸면 결과가 바뀐다

접속 주소	실행되는 조건	결과
/search?category=식품	WHERE category = '식품'	3건 (P003·P004·P005)
/search?category=가전	WHERE category = '가전'	2건 (P001·P002)
/search?category=생활용품	WHERE category = '생활용품'	3건 (P006·P007·P008)
/search?category=문구	WHERE category = '문구'	2건 (P009·P010)

상품 목록에 식품 같은 링크 네 개 = 쇼핑몰의 카테고리 메뉴

5. 파라미터 바인딩

취약한 코드 vs 안전한 코드

```
1 // ✕ 위험: 사용자 입력을 문자열로 직접 이어 붙임
2 // input 은 사용자가 화면에 입력한 값. 무엇이 들어올지 서버는 알 수 없다
3 const sql = "SELECT * FROM customers WHERE cust_id = '" + input + "'";
4 // 입력값이 ' OR '1'='1' 이면 완성되는 SQL은
5 // SELECT * FROM customers WHERE cust_id = '' OR '1'='1'
6 // → 조건이 항상 참이 되어 전체 고객 명단(이메일·포인트 포함)이 통째로 유출된다
7 // 입력값 속의 따옴표(')가 SQL 문법의 일부로 해석되는 것이 문제의 핵심이다
8
9 // ✓ 안전: 파라미터 바인딩
10 // SQL 문장은 ? 로 고정해 두고, 값은 배열로 따로 넘긴다
11 const [rows] = await pool.query(
12   'SELECT * FROM customers WHERE cust_id = ?',
13   [input]
14 );
15 // 입력값은 항상 "데이터"로만 취급되어 SQL의 구조를 바꿀 수 없다
16 // 같은 ' OR '1'='1' 을 넣어도 "그런 글자의 고객번호"를 찾을 뿐이라 0건이 나온다
```

입력값으로 SQL 구조를 조작하는 공격 = SQL 인젝션(SQL Injection)

5. 파라미터 바인딩

같은 입력 ' OR '1'='1' 의 두 결말

× 문자열 이어 붙이기

따옴표가 SQL 문법으로 해석된다

완성된 SQL	WHERE cust_id = " OR '1'='1'
조건	항상 참
결과	전체 고객 명단 유출

VS

✓ ? 바인딩

입력값은 항상 「데이터」로만 취급된다

입력값 취급	그런 글자의 고객번호(데이터)
SQL 구조	바뀌지 않는다
결과	0건

5. 파라미터 바인딩

SQL 인젝션을 막는 네 가지 습관



파라미터 바인딩을 항상 사용

사용자 입력이 SQL에 들어가는 자리는 예외 없이 ?



최소 권한 원칙

웹 서버용 DB 계정에 DROP·GRANT 권한을 주지 않는다 (10주차 DCL)



오류 메시지 숨기기

SQL 문이나 테이블 구조를 그대로 노출하지 않는다



비밀번호는 환경 변수로

접속 비밀번호를 코드에 남기지 않는 것이 실무 표준

SQL 인젝션은 기술 문제가 아니라 회사 존폐 문제 — 개인정보 유출은 손해배상·과징금·신뢰 상실로 이어진다

확인 퀴즈

사용자 입력값을 문자열 연결로 SQL에 그대로 붙였을 때, 공격자가 ' OR '1'='1 같은 입력으로 쿼리의 의미를 바꿔 버리는 공격은?

① XSS(크로스 사이트 스크립팅)

② CSRF(요청 위조)

③ SQL 인젝션

④ 스니핑

정답 및 해설

정답

③ SQL 인젝션

해설

SQL 인젝션은 입력값이 SQL 문법으로 해석되면서 발생합니다. 고객 조회 화면 하나가 뚫리면 고객 명단 전체가 유출될 수 있으므로, 입력값은 반드시 파라미터 바인딩으로 전달해야 합니다.

12주차 핵심 정리

핵심 ①	브라우저-웹 서버-DBMS 3계층, 브라우저는 DB에 직접 접속하지 않는다
핵심 ②	Express 라우트 <code>app.get('/주소', 함수)</code> 가 주소와 SQL 실행을 연결
핵심 ③	<code>pool.query()</code> 결과 rows를 HTML로 조립해 <code>res.send()</code> 로 응답
핵심 ④	검색 조건은 쿼리스트링(<code>req.query</code>)으로 받아 WHERE의 ?에 바인딩
핵심 ⑤	사용자 입력은 예외 없이 파라미터 바인딩 — SQL 인젝션 방어 기본

다음 주는 조회에 등록·수정·삭제를 더해 CRUD를 완성합니다

다음 주차 예고

13주차 · 웹과 DB 연동 2 (CRUD)

01

CRUD

업무 화면의
네 가지 기본 동작

02

등록

폼에서 받은 값을
INSERT로 저장

03

수정과 삭제

UPDATE·DELETE와
affectedRows

04

트랜잭션

주문 등록을
한 묶음으로 처리

12주차 마무리

학습 마무리

app.js에 라우트를 더해 가며 다운유통 업무 화면을 만들어 갑니다

✓ 3계층 구조와 요청 흐름

✓ /products 조회 페이지와 /search 검색

✓ Node.js·Express·mysql2 설치와 연결

✓ ? 바인딩으로 SQL 인젝션 방어

이어서는 활동

실습 문제 w12-1~9 · 퀴즈 10